

DOMAIN 4

TOOL DESIGN & MCP INTEGRATION

Complete deep-dive for this blueprint domain (18% of exam) of the Claude Certified Architect — Foundations exam: every concept with worked examples, a dedicated traps-and-nuances section explaining why the tempting wrong answers are wrong, a one-page cheat sheet, and a full bank of 118 original practice questions with answers and rationales.

Section	What's inside
1. Cheat Sheet	Golden rule, quick-reference tables, and a scenario decision checklist — read this first and last.
2. Concept Guide	Full narrative coverage of every concept tested in this domain, with worked scenarios.
3. Traps & Nuances	14 recurring wrong-answer patterns: the trap, why it's tempting, and the correct principle.
4. Practice Questions	118 original scenario-based questions, 4 options each, answer marked, rationale included.

This is an independent study aid, not an official Anthropic publication or a reproduction of actual exam questions, which Anthropic does not publish.

1 · CHEAT SHEET

Golden rule: fix the tool's interface (description, schema, error contract) before adding infrastructure (classifiers, wrappers) on top of it. Almost every Domain 4 trap reaches for infrastructure before fixing the root-cause interface problem.

Fixing tool misrouting — in order

1st	Expand & differentiate descriptions: input formats, example queries, edge cases, “prefer this when...”
2nd (only if needed)	Consider a routing classifier or merging tools — last resort, not first.

Structured tool error shape

errorCategory	e.g. RATE_LIMITED, VALIDATION_ERROR, NOT_FOUND
isRetryable	true / false — lets the coordinator decide programmatically
partialResults	Whatever was actually retrieved before failure, if anything
Bad pattern	{ "error": "Operation failed" } — gives the coordinator nothing to act on

MCP scope & transport

.mcp.json	Shared, team-wide, version-controlled servers.
~/.claude.json	Personal / experimental servers.
Local (stdio)	Single-machine, developer-local tooling.
Remote (HTTP/SSE)	Centrally maintained, team-shared resources.
Secrets	`\${ENV_VAR}` substitution, always — regardless of repo visibility.

MCP primitives

Tool	An invocable action (change or fetch live state).
Resource	Readable, largely static content/data.
Prompt	A reusable instruction template.

Decision checklist for any Domain 4 scenario

- Two tools keep getting confused? → rewrite descriptions with concrete examples before anything else.
- Tool fails and coordinator can't recover well? → the error response needs category + retryable + partial results, not a longer string.
- One tool, several modes via a flag? → split into narrow, single-purpose tools instead.
- Result set could be huge? → real pagination, not a hard cap.
- Sensitive action retried after a timeout? → idempotency key, not “no retries allowed.”

- New/unfamiliar third-party MCP server? → least privilege by default, review before broad access.

Tool Design & MCP Integration

2 · CONCEPT GUIDE

Tool descriptions are the interface the model uses to decide which tool to call — this domain tests whether you can design that interface well, configure MCP correctly, and make tool failures recoverable rather than opaque.

4.1 Tool description quality

Minimal, generic tool descriptions are a leading cause of misrouted tool calls. Two tools described only as “Gets customer information” and “Gets order details” will be confused by the model whenever an input sits near the boundary between the two, because the descriptions give it nothing to discriminate on.

Worked scenario

Production logs show `get_customer` being called for order-related queries roughly 45% of the time it shouldn't be. The two tool descriptions are minimal and nearly identical. What's the first fix to try?

Not a routing classifier bolted in front of tool selection, and **not** merging the two tools into one. The first, root-cause fix is to **expand and differentiate the descriptions**: state each tool's expected input format, give 2–3 concrete example queries it should handle, call out edge cases, and explicitly state when to prefer this tool over its neighbor. Only escalate to heavier infrastructure (a classifier, merged tools, a disambiguation step) if better descriptions don't resolve it.

General description-writing checklist the exam rewards:

- State the expected input format precisely (not just “a customer identifier” — specify which ID type).
- Give concrete example queries the tool is meant to serve.
- Name edge cases and boundary conditions explicitly.
- State when to prefer this tool over a similarly-named neighbor.
- Keep the description length proportional to how much ambiguity exists between it and other available tools — near-duplicate tools need the most differentiation.

4.2 MCP server configuration

(Overlaps Domain 2 — the exam tests it from both the Claude Code workflow angle and the pure integration-design angle.) Project-scoped servers belong in version-controlled `.mcp.json`; personal or experimental servers belong in `~/.claude.json`. Secrets always flow through environment-variable substitution (`${VAR_NAME}`), never as literal committed values.

4.3 Structured error responses from tools

What a tool returns on failure determines whether a coordinator can recover intelligently or has to give up. A tool that returns a bare string like **“Operation failed”** gives the calling agent nothing actionable. A tool that returns a structured error object gives the coordinator enough information to make a real decision.

```
// Weak – coordinator can't act on this
{ "error": "Operation failed" }

// Strong – coordinator has options
{
```

```
"errorCategory": "RATE_LIMITED",
"isRetryable": true,
"attemptedQuery": { "orderId": "A-4471" },
"partialResults": null,
"retryAfterMs": 2000
}
```

With the structured version, the coordinator can decide to retry, fall back to an alternative tool or data source, or annotate the gap in its final output and continue — rather than failing the entire task on one opaque error. Exam distractors here are almost always generic status messages (“error”, “failed”, HTTP codes with no category or retryability signal) that look reasonable until you ask what the coordinator can actually do with them.

4.4 Tool design checklist

- Differentiate near-duplicate tools with concrete examples and explicit “prefer this when...” guidance.
- Keep each tool's scope narrow and single-purpose rather than one tool with many optional modes.
- Return structured, categorized errors (category, retryable flag, partial results) instead of bare failure strings.
- Version-control shared MCP config; keep secrets out of committed files via env-var substitution.
- Prefer fixing descriptions and error contracts over adding a routing/classification layer on top.

3 · TRAPS & NUANCES

Each card below names a specific concept, describes the wrong answer the exam is likely to dangle in front of you, explains why that wrong answer feels right, and states the principle that actually resolves it. Read this section right before the exam as a final refresh.

CONCEPT: Tool misrouting fixes

The trap: Building a routing classifier in front of tool selection as the first response to misrouting.

Why it's tempting: A classifier sounds like the more sophisticated, senior-engineer solution to an ambiguity problem.

The correct principle: The root cause of most misrouting is under-differentiated tool descriptions — fix those first; only add classifier infrastructure if better descriptions genuinely don't resolve it.

CONCEPT: Structured tool errors

The trap: “Fixing” a bare error string by making the message longer or more descriptive, while keeping it as a single text field.

Why it's tempting: A longer message feels like it adds useful detail.

The correct principle: A useful error needs distinct structured fields — category, retryability, partial results — not just more prose in one field; a coordinator needs to parse it programmatically, not read it.

CONCEPT: MCP config scope vs. credential type

The trap: Assuming a server needing personal credentials belongs in `~/.claude.json` regardless of whether the team shares it.

Why it's tempting: Credentials feel inherently personal, so it's easy to conflate credential ownership with config scope.

The correct principle: Config location should match sharing intent (is this server used by the whole team or just me), not credential type — a shared server with per-user credentials still belongs in `.mcp.json`, with the credential itself injected via environment variable.

CONCEPT: Secrets and repo visibility

The trap: Assuming it's acceptable to commit a literal secret because the repository is private.

Why it's tempting: “Private repo” sounds like a sufficient security boundary on its own.

The correct principle: Environment-variable substitution should be used regardless of repo visibility — private repos still have multiple collaborators, forks, and CI logs that can leak a committed secret.

CONCEPT: Tool scope: single-purpose vs. multi-mode

The trap: Adding a “mode” or “action” parameter to an existing tool instead of splitting it into separate tools.

Why it's tempting: One flexible tool feels like it avoids cluttering the toolset with near-duplicates.

The correct principle: Narrow, single-purpose tools are easier for a model to select correctly than one tool whose behavior depends on a flag the model has to set correctly every time.

CONCEPT: MCP transport choice

The trap: Defaulting every MCP server to remote/HTTP “for consistency” across the team's tooling.

Why it's tempting: Uniform infrastructure feels simpler to reason about and maintain.

The correct principle: Transport should match the actual sharing need: local (stdio) for single-machine developer tooling, remote (HTTP/SSE) for centrally maintained, team-shared resources.

CONCEPT: Ambiguous tool input schemas

The trap: Adding a free-text “notes” or “context” field to a tool's schema to handle cases the existing parameters don't cleanly cover.

Why it's tempting: A free-text escape hatch feels like it gives the model flexibility to express intent.

The correct principle: A free-text field papers over a design problem — the right fix is usually to make the ambiguous parameter itself precise (e.g., separate ID types, explicit units), not to add an unstructured workaround.

CONCEPT: Pagination design

The trap: Hardcoding a small result cap on a tool with no way to retrieve additional results.

Why it's tempting: A hard cap feels like a quick, complete fix for the risk of unbounded results.

The correct principle: Unbounded results should be handled with real pagination (cursor or offset parameters) and a sensible default limit — a hard cap silently makes some data permanently unreachable.

CONCEPT: Credential scoping across tools

The trap: Sharing one broad credential across multiple tools with very different risk profiles, “to keep things simple.”

Why it's tempting: Fewer credentials to manage feels like lower operational overhead.

The correct principle: Each tool's credentials should be scoped narrowly to what that specific tool needs — sharing broad credentials means a low-risk tool's compromise can affect a high-risk one.

CONCEPT: Idempotency vs. retry avoidance

The trap: Preventing all retries on a sensitive tool (like a payment call) to avoid duplicate effects.

Why it's tempting: Removing retries entirely feels like the simplest way to eliminate the duplication risk.

The correct principle: The better fix is making the operation idempotent (e.g., via an idempotency key) so a legitimate retry after a transient failure doesn't duplicate the effect — losing retry resilience trades one problem for another.

CONCEPT: Testing before rollout

The trap: Shipping a new or modified MCP server / tool broadly and monitoring for complaints as the validation method.

Why it's tempting: Shipping fast and reacting to issues feels like it gets value to users sooner.

The correct principle: Tool and MCP server changes should be tested against representative queries and edge cases — including failure behavior — before a broad rollout, not validated after the fact by user reports.

CONCEPT: Tool deprecation

The trap: Deleting an old tool the moment a replacement is available.

Why it's tempting: Removing the old version immediately avoids the overhead of maintaining two versions in parallel.

The correct principle: Widely-used tools need staged deprecation — introduce the replacement, mark the old one deprecated, and remove it only after callers have had time to migrate.

CONCEPT: Third-party MCP server trust

The trap: Granting a new, unfamiliar third-party MCP server broad access up front “to avoid setup friction,” planning to narrow permissions later.

Why it's tempting: Broad access avoids repeated permission prompts during initial setup and testing.

The correct principle: Third-party servers should default to least privilege from the start, especially when unfamiliar — “narrow it later” access rarely actually gets revisited once the integration is working.

CONCEPT: Choosing the right MCP primitive

The trap: Exposing a static, read-only reference document as a “tool” because tools are the most familiar MCP concept.

Why it's tempting: Tools are usually the first (and sometimes only) MCP primitive people reach for.

The correct principle: MCP distinguishes tools (invocable actions), resources (readable content), and prompts (reusable templates) — a static document is a resource, not a tool, and mismatching the primitive muddies the server's interface.

4 · PRACTICE QUESTIONS (118)

Q1. Production logs show `get_customer` being called for order-related queries roughly 45% of the time it shouldn't be. Both `get_customer` and `get_order`'s descriptions are minimal and nearly identical. What's the first fix to try?

- A. Build a routing classifier in front of tool selection.
- B. Merge `get_customer` and `get_order` into a single combined tool.
- C. Increase the model's temperature to reduce repetitive misrouting.
- D. ✓ Expand and differentiate the descriptions of `get_customer` and `get_order` — add input formats, concrete example queries, and explicit "prefer this when..." guidance.

Correct answer: D. The root cause is under-differentiated descriptions; fix that first before adding classifier infrastructure or merging tools on top of the actual problem.

Q2. Production logs show `lookup_user` being called for support-ticket queries roughly 38% of the time it shouldn't be. Both `lookup_user` and `lookup_ticket`'s descriptions are minimal and nearly identical. What's the first fix to try?

- A. Merge `lookup_user` and `lookup_ticket` into a single combined tool.
- B. Increase the model's temperature to reduce repetitive misrouting.
- C. ✓ Expand and differentiate the descriptions of `lookup_user` and `lookup_ticket` — add input formats, concrete example queries, and explicit "prefer this when..." guidance.
- D. Build a routing classifier in front of tool selection.

Correct answer: C. The root cause is under-differentiated descriptions; fix that first before adding classifier infrastructure or merging tools on top of the actual problem.

Q3. Production logs show `fetch_invoice` being called for payment-related queries roughly 41% of the time it shouldn't be. Both `fetch_invoice` and `fetch_payment`'s descriptions are minimal and nearly identical. What's the first fix to try?

- A. Increase the model's temperature to reduce repetitive misrouting.
- B. ✓ Expand and differentiate the descriptions of `fetch_invoice` and `fetch_payment` — add input formats, concrete example queries, and explicit "prefer this when..." guidance.
- C. Build a routing classifier in front of tool selection.
- D. Merge `fetch_invoice` and `fetch_payment` into a single combined tool.

Correct answer: B. The root cause is under-differentiated descriptions; fix that first before adding classifier infrastructure or merging tools on top of the actual problem.

Q4. Production logs show `get_product` being called for inventory queries roughly 33% of the time it shouldn't be. Both `get_product` and `get_inventory`'s descriptions are minimal and nearly identical. What's the first fix to try?

- A. ✓ Expand and differentiate the descriptions of `get_product` and `get_inventory` — add input formats, concrete example queries, and explicit "prefer this when..." guidance.
- B. Build a routing classifier in front of tool selection.
- C. Merge `get_product` and `get_inventory` into a single combined tool.
- D. Increase the model's temperature to reduce repetitive misrouting.

Correct answer: A. The root cause is under-differentiated descriptions; fix that first before adding classifier infrastructure or merging tools on top of the actual problem.

Q5. Production logs show search_docs being called for FAQ-type queries roughly 50% of the time it shouldn't be. Both search_docs and search_faq's descriptions are minimal and nearly identical. What's the first fix to try?

- A. Build a routing classifier in front of tool selection.
- B. Merge search_docs and search_faq into a single combined tool.
- C. Increase the model's temperature to reduce repetitive misrouting.
- D. ✓ Expand and differentiate the descriptions of search_docs and search_faq — add input formats, concrete example queries, and explicit "prefer this when..." guidance.

Correct answer: D. The root cause is under-differentiated descriptions; fix that first before adding classifier infrastructure or merging tools on top of the actual problem.

Q6. Production logs show get_employee being called for department-level queries roughly 29% of the time it shouldn't be. Both get_employee and get_department's descriptions are minimal and nearly identical. What's the first fix to try?

- A. Merge get_employee and get_department into a single combined tool.
- B. Increase the model's temperature to reduce repetitive misrouting.
- C. ✓ Expand and differentiate the descriptions of get_employee and get_department — add input formats, concrete example queries, and explicit "prefer this when..." guidance.
- D. Build a routing classifier in front of tool selection.

Correct answer: C. The root cause is under-differentiated descriptions; fix that first before adding classifier infrastructure or merging tools on top of the actual problem.

Q7. Production logs show fetch_shipment being called for order-status queries roughly 44% of the time it shouldn't be. Both fetch_shipment and fetch_order's descriptions are minimal and nearly identical. What's the first fix to try?

- A. Increase the model's temperature to reduce repetitive misrouting.
- B. ✓ Expand and differentiate the descriptions of fetch_shipment and fetch_order — add input formats, concrete example queries, and explicit "prefer this when..." guidance.
- C. Build a routing classifier in front of tool selection.
- D. Merge fetch_shipment and fetch_order into a single combined tool.

Correct answer: B. The root cause is under-differentiated descriptions; fix that first before adding classifier infrastructure or merging tools on top of the actual problem.

Q8. Production logs show get_account being called for subscription queries roughly 36% of the time it shouldn't be. Both get_account and get_subscription's descriptions are minimal and nearly identical. What's the first fix to try?

- A. ✓ Expand and differentiate the descriptions of get_account and get_subscription — add input formats, concrete example queries, and explicit "prefer this when..." guidance.
- B. Build a routing classifier in front of tool selection.
- C. Merge get_account and get_subscription into a single combined tool.
- D. Increase the model's temperature to reduce repetitive misrouting.

Correct answer: A. The root cause is under-differentiated descriptions; fix that first before adding classifier infrastructure or merging tools on top of the actual problem.

Q9. Production logs show search_contacts being called for company-level queries roughly 40% of the time it shouldn't be. Both search_contacts and search_companies's descriptions are minimal and nearly identical. What's the first fix to try?

- A. Build a routing classifier in front of tool selection.
- B. Merge search_contacts and search_companies into a single combined tool.
- C. Increase the model's temperature to reduce repetitive misrouting.
- D. ✓ Expand and differentiate the descriptions of search_contacts and search_companies — add input formats, concrete example queries, and explicit "prefer this when..." guidance.

Correct answer: D. The root cause is under-differentiated descriptions; fix that first before adding classifier infrastructure or merging tools on top of the actual problem.

Q10. Production logs show `get_ticket` being called for customer-profile queries roughly 37% of the time it shouldn't be. Both `get_ticket` and `get_customer`'s descriptions are minimal and nearly identical. What's the first fix to try?

- A. Merge `get_ticket` and `get_customer` into a single combined tool.
- B. Increase the model's temperature to reduce repetitive misrouting.
- C. ✓ Expand and differentiate the descriptions of `get_ticket` and `get_customer` — add input formats, concrete example queries, and explicit "prefer this when..." guidance.
- D. Build a routing classifier in front of tool selection.

Correct answer: C. The root cause is under-differentiated descriptions; fix that first before adding classifier infrastructure or merging tools on top of the actual problem.

Q11. A payment-processing tool fails with a `RATE_LIMITED`-type error. What should its error response look like to be useful to a coordinator?

- A. A plain string description with no structured fields at all
- B. ✓ { "errorCategory": "RATE_LIMITED", "isRetryable": true, plus any partial results }
- C. { "error": "Operation failed" }
- D. An HTTP status code with no further detail

Correct answer: B. A structured error — category, retryability, and any partial results — gives the coordinator concrete grounds to retry, fall back, or escalate; a bare string or status code doesn't.

Q12. An inventory-lookup tool fails with a `NOT_FOUND`-type error. What should its error response look like to be useful to a coordinator?

- A. ✓ { "errorCategory": "NOT_FOUND", "isRetryable": false, plus any partial results }
- B. { "error": "Operation failed" }
- C. An HTTP status code with no further detail
- D. A plain string description with no structured fields at all

Correct answer: A. A structured error — category, retryability, and any partial results — gives the coordinator concrete grounds to retry, fall back, or escalate; a bare string or status code doesn't.

Q13. A shipping-label tool fails with a `VALIDATION_ERROR`-type error. What should its error response look like to be useful to a coordinator?

- A. { "error": "Operation failed" }
- B. An HTTP status code with no further detail
- C. A plain string description with no structured fields at all
- D. ✓ { "errorCategory": "VALIDATION_ERROR", "isRetryable": true, plus any partial results }

Correct answer: D. A structured error — category, retryability, and any partial results — gives the coordinator concrete grounds to retry, fall back, or escalate; a bare string or status code doesn't.

Q14. A search tool fails with a `TIMEOUT`-type error. What should its error response look like to be useful to a coordinator?

- A. An HTTP status code with no further detail
- B. A plain string description with no structured fields at all
- C. ✓ { "errorCategory": "TIMEOUT", "isRetryable": true, plus any partial results }
- D. { "error": "Operation failed" }

Correct answer: C. A structured error — category, retryability, and any partial results — gives the coordinator concrete grounds to retry, fall back, or escalate; a bare string or status code doesn't.

Q15. A `crm-update` tool fails with a `PERMISSION_DENIED`-type error. What should its error response look like to be useful to a coordinator?

- A. A plain string description with no structured fields at all
- B. ✓ { "errorCategory": "PERMISSION_DENIED", "isRetryable": false, plus any partial results }
- C. { "error": "Operation failed" }
- D. An HTTP status code with no further detail

Correct answer: B. A structured error — category, retryability, and any partial results — gives the coordinator concrete grounds to retry, fall back, or escalate; a bare string or status code doesn't.

Q16. A document-generation tool fails with a `UPSTREAM_UNAVAILABLE`-type error. What should its error response look like to be useful to a coordinator?

- A. ✓ { "errorCategory": "UPSTREAM_UNAVAILABLE", "isRetryable": true, plus any partial results }
- B. { "error": "Operation failed" }
- C. An HTTP status code with no further detail
- D. A plain string description with no structured fields at all

Correct answer: A. A structured error — category, retryability, and any partial results — gives the coordinator concrete grounds to retry, fall back, or escalate; a bare string or status code doesn't.

Q17. A booking tool fails with a `CONFLICT`-type error. What should its error response look like to be useful to a coordinator?

- A. { "error": "Operation failed" }
- B. An HTTP status code with no further detail
- C. A plain string description with no structured fields at all
- D. ✓ { "errorCategory": "CONFLICT", "isRetryable": false, plus any partial results }

Correct answer: D. A structured error — category, retryability, and any partial results — gives the coordinator concrete grounds to retry, fall back, or escalate; a bare string or status code doesn't.

Q18. A file-upload tool fails with a `QUOTA_EXCEEDED`-type error. What should its error response look like to be useful to a coordinator?

- A. An HTTP status code with no further detail
- B. A plain string description with no structured fields at all
- C. ✓ { "errorCategory": "QUOTA_EXCEEDED", "isRetryable": true, plus any partial results }
- D. { "error": "Operation failed" }

Correct answer: C. A structured error — category, retryability, and any partial results — gives the coordinator concrete grounds to retry, fall back, or escalate; a bare string or status code doesn't.

Q19. Where should the MCP configuration for a shared internal database MCP server the whole engineering team relies on live?

- A. A hardcoded value in the tool's source code
- B. ✓ `.mcp.json` (project-level, version-controlled)
- C. `~/.claude.json` (personal, not shared)
- D. Inside `CLAUDE.md` as free text

Correct answer: B. Shared, team-wide servers belong in version-controlled `.mcp.json`; personal or experimental servers belong in the non-shared `~/.claude.json`.

Q20. Where should the MCP configuration for a personal experimental MCP server one engineer is trying out before proposing it live?

- A. ✓ `~/.claude.json` (personal, not shared)
- B. `.mcp.json` (project-level, version-controlled)
- C. Inside `CLAUDE.md` as free text
- D. A hardcoded value in the tool's source code

Correct answer: A. Shared, team-wide servers belong in version-controlled `.mcp.json`; personal or experimental servers belong in the non-shared `~/.claude.json`.

Q21. Where should the MCP configuration for a shared CI-status MCP server every contributor needs live?

- A. `~/claude.json` (personal, not shared)
- B. Inside `CLAUDE.md` as free text
- C. A hardcoded value in the tool's source code
- D. ✓ `.mcp.json` (project-level, version-controlled)

Correct answer: D. Shared, team-wide servers belong in version-controlled `.mcp.json`; personal or experimental servers belong in the non-shared `~/claude.json`.

Q22. Where should the MCP configuration for a personal calendar MCP server unrelated to the shared project live?

- A. Inside `CLAUDE.md` as free text
- B. A hardcoded value in the tool's source code
- C. ✓ `~/claude.json` (personal, not shared)
- D. `.mcp.json` (project-level, version-controlled)

Correct answer: C. Shared, team-wide servers belong in version-controlled `.mcp.json`; personal or experimental servers belong in the non-shared `~/claude.json`.

Q23. Where should the MCP configuration for a shared ticketing-system MCP server required by the whole team's workflow live?

- A. A hardcoded value in the tool's source code
- B. ✓ `.mcp.json` (project-level, version-controlled)
- C. `~/claude.json` (personal, not shared)
- D. Inside `CLAUDE.md` as free text

Correct answer: B. Shared, team-wide servers belong in version-controlled `.mcp.json`; personal or experimental servers belong in the non-shared `~/claude.json`.

Q24. Where should the MCP configuration for a personal note-taking MCP server used only by one developer live?

- A. ✓ `~/claude.json` (personal, not shared)
- B. `.mcp.json` (project-level, version-controlled)
- C. Inside `CLAUDE.md` as free text
- D. A hardcoded value in the tool's source code

Correct answer: A. Shared, team-wide servers belong in version-controlled `.mcp.json`; personal or experimental servers belong in the non-shared `~/claude.json`.

Q25. Where should the MCP configuration for a shared search-index MCP server the repo depends on for every contributor live?

- A. `~/claude.json` (personal, not shared)
- B. Inside `CLAUDE.md` as free text
- C. A hardcoded value in the tool's source code
- D. ✓ `.mcp.json` (project-level, version-controlled)

Correct answer: D. Shared, team-wide servers belong in version-controlled `.mcp.json`; personal or experimental servers belong in the non-shared `~/claude.json`.

Q26. Where should the MCP configuration for a personal weather MCP server with no relation to the team's actual work live?

- A. Inside `CLAUDE.md` as free text
- B. A hardcoded value in the tool's source code
- C. ✓ `~/claude.json` (personal, not shared)
- D. `.mcp.json` (project-level, version-controlled)

Correct answer: C. Shared, team-wide servers belong in version-controlled `.mcp.json`; personal or experimental servers belong in the non-shared `~/claude.json`.

Q27. A team needs to supply credentials for a payments MCP server inside a version-controlled MCP config file. What's correct?

- A. Store the key in a code comment near the config.
- B. ✓ Use environment-variable substitution, e.g. `${API_KEY}`, so no literal secret is committed to the config file.
- C. Commit the literal API key directly into the config for simplicity.
- D. Base64-encode the key and commit the encoded value.

Correct answer: B. Environment-variable substitution keeps secrets out of committed files entirely, regardless of how they're encoded — encoding is not the same as not committing the secret.

Q28. A team needs to supply credentials for a CRM MCP server inside a version-controlled MCP config file. What's correct?

- A. ✓ Use environment-variable substitution, e.g. `${API_KEY}`, so no literal secret is committed to the config file.
- B. Commit the literal API key directly into the config for simplicity.
- C. Base64-encode the key and commit the encoded value.
- D. Store the key in a code comment near the config.

Correct answer: A. Environment-variable substitution keeps secrets out of committed files entirely, regardless of how they're encoded — encoding is not the same as not committing the secret.

Q29. A team needs to supply credentials for a cloud-storage MCP server inside a version-controlled MCP config file. What's correct?

- A. Commit the literal API key directly into the config for simplicity.
- B. Base64-encode the key and commit the encoded value.
- C. Store the key in a code comment near the config.
- D. ✓ Use environment-variable substitution, e.g. `${API_KEY}`, so no literal secret is committed to the config file.

Correct answer: D. Environment-variable substitution keeps secrets out of committed files entirely, regardless of how they're encoded — encoding is not the same as not committing the secret.

Q30. A team needs to supply credentials for an analytics MCP server inside a version-controlled MCP config file. What's correct?

- A. Base64-encode the key and commit the encoded value.
- B. Store the key in a code comment near the config.
- C. ✓ Use environment-variable substitution, e.g. `${API_KEY}`, so no literal secret is committed to the config file.
- D. Commit the literal API key directly into the config for simplicity.

Correct answer: C. Environment-variable substitution keeps secrets out of committed files entirely, regardless of how they're encoded — encoding is not the same as not committing the secret.

Q31. A team needs to supply credentials for an internal database MCP server inside a version-controlled MCP config file. What's correct?

- A. Store the key in a code comment near the config.
- B. ✓ Use environment-variable substitution, e.g. `${API_KEY}`, so no literal secret is committed to the config file.
- C. Commit the literal API key directly into the config for simplicity.
- D. Base64-encode the key and commit the encoded value.

Correct answer: B. Environment-variable substitution keeps secrets out of committed files entirely, regardless of how they're encoded — encoding is not the same as not committing the secret.

Q32. A team needs to supply credentials for a ticketing MCP server inside a version-controlled MCP config file. What's correct?

- A. ✓ Use environment-variable substitution, e.g. `${API_KEY}`, so no literal secret is committed to the config file.
- B. Commit the literal API key directly into the config for simplicity.
- C. Base64-encode the key and commit the encoded value.
- D. Store the key in a code comment near the config.

Correct answer: A. Environment-variable substitution keeps secrets out of committed files entirely, regardless of how they're encoded — encoding is not the same as not committing the secret.

Q33. Two tools — a tool that searches by exact ID and a tool that searches by fuzzy name match — are being confused by the model in production. What's the highest-leverage description improvement?

- A. Give both tools identical descriptions to keep things consistent.
- B. Remove one of the two tools to eliminate the ambiguity.
- C. Rename both tools to nearly identical names for symmetry.
- D. ✓ Add concrete example queries for each, showing when an exact ID is available versus when only a name is known to each tool's description.

Correct answer: D. The model relies entirely on descriptions to discriminate between similar tools; concrete, contrasting guidance in each description is what actually reduces misrouting.

Q34. Two tools — a tool that reads a single record and a tool that lists many records — are being confused by the model in production. What's the highest-leverage description improvement?

- A. Remove one of the two tools to eliminate the ambiguity.
- B. Rename both tools to nearly identical names for symmetry.
- C. ✓ Add explicit guidance on singular vs. plural use cases and expected result size to each tool's description.
- D. Give both tools identical descriptions to keep things consistent.

Correct answer: C. The model relies entirely on descriptions to discriminate between similar tools; concrete, contrasting guidance in each description is what actually reduces misrouting.

Q35. Two tools — a tool that creates a resource and a tool that updates an existing one — are being confused by the model in production. What's the highest-leverage description improvement?

- A. Rename both tools to nearly identical names for symmetry.
- B. ✓ Add a clear statement of the precondition each expects (resource must not exist vs. must already exist) to each tool's description.
- C. Give both tools identical descriptions to keep things consistent.
- D. Remove one of the two tools to eliminate the ambiguity.

Correct answer: B. The model relies entirely on descriptions to discriminate between similar tools; concrete, contrasting guidance in each description is what actually reduces misrouting.

Q36. Two tools — a tool that queries live data and a tool that queries a cached snapshot — are being confused by the model in production. What's the highest-leverage description improvement?

- A. ✓ Add an explicit note on staleness tolerance and when live data is required to each tool's description.
- B. Give both tools identical descriptions to keep things consistent.
- C. Remove one of the two tools to eliminate the ambiguity.
- D. Rename both tools to nearly identical names for symmetry.

Correct answer: A. The model relies entirely on descriptions to discriminate between similar tools; concrete, contrasting guidance in each description is what actually reduces misrouting.

Q37. Two tools — a tool scoped to one region and a tool scoped globally — are being confused by the model in production. What's the highest-leverage description improvement?

- A. Give both tools identical descriptions to keep things consistent.
- B. Remove one of the two tools to eliminate the ambiguity.
- C. Rename both tools to nearly identical names for symmetry.
- D. ✓ Add explicit input-format guidance showing which region parameter each expects to each tool's description.

Correct answer: D. The model relies entirely on descriptions to discriminate between similar tools; concrete, contrasting guidance in each description is what actually reduces misrouting.

Q38. Two tools — a tool for synchronous lookups and a tool for kicking off an async job — are being confused by the model in production. What's the highest-leverage description improvement?

- A. Remove one of the two tools to eliminate the ambiguity.
- B. Rename both tools to nearly identical names for symmetry.
- C. ✓ Add explicit guidance on expected response time and when to prefer polling to each tool's description.
- D. Give both tools identical descriptions to keep things consistent.

Correct answer: C. The model relies entirely on descriptions to discriminate between similar tools; concrete, contrasting guidance in each description is what actually reduces misrouting.

Q39. Two tools — a tool that returns summary statistics and a tool that returns raw records — are being confused by the model in production. What's the highest-leverage description improvement?

- A. Rename both tools to nearly identical names for symmetry.
- B. ✓ Add example queries showing when aggregate vs. granular data is actually needed to each tool's description.
- C. Give both tools identical descriptions to keep things consistent.
- D. Remove one of the two tools to eliminate the ambiguity.

Correct answer: B. The model relies entirely on descriptions to discriminate between similar tools; concrete, contrasting guidance in each description is what actually reduces misrouting.

Q40. Two tools — a tool for internal employees and a tool for external customers — are being confused by the model in production. What's the highest-leverage description improvement?

- A. ✓ Add an explicit statement of which caller context each tool is meant for to each tool's description.
- B. Give both tools identical descriptions to keep things consistent.
- C. Remove one of the two tools to eliminate the ambiguity.
- D. Rename both tools to nearly identical names for symmetry.

Correct answer: A. The model relies entirely on descriptions to discriminate between similar tools; concrete, contrasting guidance in each description is what actually reduces misrouting.

Q41. A tool design uses a single tool with a mode flag that can create, update, delete, or list resources depending on the flag's value. Is this good practice for tool design?

- A. Keep the single multi-mode tool; fewer tools is always simpler for the model.
- B. Add more optional parameters to the same tool instead of splitting it.
- C. Leave the design as-is and rely on the system prompt to clarify usage.
- D. ✓ Split it into separate, single-purpose tools — each with a clear, narrow scope the model can select unambiguously.

Correct answer: D. Narrow, single-purpose tools are easier for a model to select correctly than one bloated tool whose behavior depends on a mode flag the model must set correctly every time.

Q42. A tool design uses a single tool that both sends emails and schedules calendar events depending on a type parameter. Is this good practice for tool design?

- A. Add more optional parameters to the same tool instead of splitting it.
- B. Leave the design as-is and rely on the system prompt to clarify usage.
- C. ✓ Split it into separate, single-purpose tools — each with a clear, narrow scope the model can select unambiguously.
- D. Keep the single multi-mode tool; fewer tools is always simpler for the model.

Correct answer: C. Narrow, single-purpose tools are easier for a model to select correctly than one bloated tool whose behavior depends on a mode flag the model must set correctly every time.

Q43. A tool design uses a single tool that both reads and writes configuration depending on an action parameter. Is this good practice for tool design?

- A. Leave the design as-is and rely on the system prompt to clarify usage.
- B. ✓ Split it into separate, single-purpose tools — each with a clear, narrow scope the model can select unambiguously.
- C. Keep the single multi-mode tool; fewer tools is always simpler for the model.
- D. Add more optional parameters to the same tool instead of splitting it.

Correct answer: B. Narrow, single-purpose tools are easier for a model to select correctly than one bloated tool whose behavior depends on a mode flag the model must set correctly every time.

Q44. A tool design uses a single tool that handles both customer lookups and order lookups depending on a target parameter. Is this good practice for tool design?

- A. ✓ Split it into separate, single-purpose tools — each with a clear, narrow scope the model can select unambiguously.
- B. Keep the single multi-mode tool; fewer tools is always simpler for the model.
- C. Add more optional parameters to the same tool instead of splitting it.
- D. Leave the design as-is and rely on the system prompt to clarify usage.

Correct answer: A. Narrow, single-purpose tools are easier for a model to select correctly than one bloated tool whose behavior depends on a mode flag the model must set correctly every time.

Q45. A tool design uses a single tool that both validates and submits a form depending on a mode flag. Is this good practice for tool design?

- A. Keep the single multi-mode tool; fewer tools is always simpler for the model.
- B. Add more optional parameters to the same tool instead of splitting it.
- C. Leave the design as-is and rely on the system prompt to clarify usage.
- D. ✓ Split it into separate, single-purpose tools — each with a clear, narrow scope the model can select unambiguously.

Correct answer: D. Narrow, single-purpose tools are easier for a model to select correctly than one bloated tool whose behavior depends on a mode flag the model must set correctly every time.

Q46. A tool design uses a single tool that both queries and mutates inventory depending on an operation parameter. Is this good practice for tool design?

- A. Add more optional parameters to the same tool instead of splitting it.
- B. Leave the design as-is and rely on the system prompt to clarify usage.
- C. ✓ Split it into separate, single-purpose tools — each with a clear, narrow scope the model can select unambiguously.
- D. Keep the single multi-mode tool; fewer tools is always simpler for the model.

Correct answer: C. Narrow, single-purpose tools are easier for a model to select correctly than one bloated tool whose behavior depends on a mode flag the model must set correctly every time.

Q47. For a local development tool that only needs to run on the developer's own machine, which MCP server transport generally fits better?

- A. A local stdio server for a centrally shared, team-wide resource
- B. ✓ A local (stdio) MCP server
- C. Either transport works identically with no meaningful tradeoff
- D. A remote server for purely local, single-machine tooling

Correct answer: B. Local (stdio) servers suit single-machine, developer-local tooling; remote (HTTP/SSE) servers suit centrally maintained, team-shared resources reachable by many sessions.

Q48. For a shared team resource that must be reachable by every contributor's session, which MCP server transport generally fits better?

- A. ✓ A remote (HTTP/SSE) MCP server
- B. Either transport works identically with no meaningful tradeoff
- C. A remote server for purely local, single-machine tooling
- D. A local stdio server for a centrally shared, team-wide resource

Correct answer: A. Local (stdio) servers suit single-machine, developer-local tooling; remote (HTTP/SSE) servers suit centrally maintained, team-shared resources reachable by many sessions.

Q49. For a tool wrapping a local file-system operation, which MCP server transport generally fits better?

- A. Either transport works identically with no meaningful tradeoff
- B. A remote server for purely local, single-machine tooling
- C. A local stdio server for a centrally shared, team-wide resource
- D. ✓ A local (stdio) MCP server

Correct answer: D. Local (stdio) servers suit single-machine, developer-local tooling; remote (HTTP/SSE) servers suit centrally maintained, team-shared resources reachable by many sessions.

Q50. For a tool that must be centrally maintained and updated without every developer reinstalling anything, which MCP server transport generally fits better?

- A. A remote server for purely local, single-machine tooling
- B. A local stdio server for a centrally shared, team-wide resource
- C. ✓ A remote (HTTP/SSE) MCP server
- D. Either transport works identically with no meaningful tradeoff

Correct answer: C. Local (stdio) servers suit single-machine, developer-local tooling; remote (HTTP/SSE) servers suit centrally maintained, team-shared resources reachable by many sessions.

Q51. For a tool used only during a single developer's local debugging session, which MCP server transport generally fits better?

- A. A local stdio server for a centrally shared, team-wide resource
- B. ✓ A local (stdio) MCP server
- C. Either transport works identically with no meaningful tradeoff
- D. A remote server for purely local, single-machine tooling

Correct answer: B. Local (stdio) servers suit single-machine, developer-local tooling; remote (HTTP/SSE) servers suit centrally maintained, team-shared resources reachable by many sessions.

Q52. For a tool that needs centralized authentication and rate-limiting shared across a whole team, which MCP server transport generally fits better?

- A. ✓ A remote (HTTP/SSE) MCP server
- B. Either transport works identically with no meaningful tradeoff
- C. A remote server for purely local, single-machine tooling
- D. A local stdio server for a centrally shared, team-wide resource

Correct answer: A. Local (stdio) servers suit single-machine, developer-local tooling; remote (HTTP/SSE) servers suit centrally maintained, team-shared resources reachable by many sessions.

Q53. A tool's input schema has a date parameter that accepts free-form text like "next Tuesday", leading to inconsistent or wrong calls. What's the right fix?

- A. Leave the ambiguous parameter as-is and document it only in a comment.
- B. Add a free-text "notes" field for the model to clarify intent instead of fixing the schema.
- C. Remove the parameter entirely rather than clarifying it.
- D. ✓ Redesign the input schema to use a strict ISO-8601 date format with clear validation.

Correct answer: D. Ambiguous or underspecified input schemas are a direct source of tool-calling errors — precise, unambiguous parameter design prevents the model from having to guess intent.

Q54. A tool's input schema has an amount parameter with no currency specified, leading to inconsistent or wrong calls. What's the right fix?

- A. Add a free-text "notes" field for the model to clarify intent instead of fixing the schema.
- B. Remove the parameter entirely rather than clarifying it.
- C. ✓ Redesign the input schema to use an explicit currency field alongside the amount.
- D. Leave the ambiguous parameter as-is and document it only in a comment.

Correct answer: C. Ambiguous or underspecified input schemas are a direct source of tool-calling errors — precise, unambiguous parameter design prevents the model from having to guess intent.

Q55. A tool's input schema has an ID parameter that could be either an internal ID or an external reference number, leading to inconsistent or wrong calls. What's the right fix?

- A. Remove the parameter entirely rather than clarifying it.
- B. ✓ Redesign the input schema to use two distinct, clearly-named parameters for each ID type.
- C. Leave the ambiguous parameter as-is and document it only in a comment.
- D. Add a free-text "notes" field for the model to clarify intent instead of fixing the schema.

Correct answer: B. Ambiguous or underspecified input schemas are a direct source of tool-calling errors — precise, unambiguous parameter design prevents the model from having to guess intent.

Q56. A tool's input schema has a free-text "query" parameter used for both search and exact lookup, leading to inconsistent or wrong calls. What's the right fix?

- A. ✓ Redesign the input schema to use separate parameters distinguishing search terms from exact identifiers.
- B. Leave the ambiguous parameter as-is and document it only in a comment.
- C. Add a free-text "notes" field for the model to clarify intent instead of fixing the schema.
- D. Remove the parameter entirely rather than clarifying it.

Correct answer: A. Ambiguous or underspecified input schemas are a direct source of tool-calling errors — precise, unambiguous parameter design prevents the model from having to guess intent.

Q57. A tool's input schema has a boolean flag whose meaning isn't obvious from its name alone, leading to inconsistent or wrong calls. What's the right fix?

- A. Leave the ambiguous parameter as-is and document it only in a comment.
- B. Add a free-text "notes" field for the model to clarify intent instead of fixing the schema.
- C. Remove the parameter entirely rather than clarifying it.
- D. ✓ Redesign the input schema to use a clearly-named flag with an explicit description of what true/false means.

Correct answer: D. Ambiguous or underspecified input schemas are a direct source of tool-calling errors — precise, unambiguous parameter design prevents the model from having to guess intent.

Q58. A tool's input schema has a units-less numeric parameter for a physical quantity, leading to inconsistent or wrong calls. What's the right fix?

- A. Add a free-text "notes" field for the model to clarify intent instead of fixing the schema.
- B. Remove the parameter entirely rather than clarifying it.
- C. ✓ Redesign the input schema to use an explicit units field or unit-suffixed parameter name.
- D. Leave the ambiguous parameter as-is and document it only in a comment.

Correct answer: C. Ambiguous or underspecified input schemas are a direct source of tool-calling errors — precise, unambiguous parameter design prevents the model from having to guess intent.

Q59. A search tool that can return thousands of matching records in one call. How should this tool's design handle potentially very large result sets?

- A. Leave result size unbounded and rely on the model to only ask for what it needs.
- B. ✓ Add pagination (page size and cursor/offset parameters) and a sensible default limit, rather than returning unbounded results.
- C. Let the tool always return every matching result in a single call.
- D. Cap results at a hardcoded small number with no way to retrieve more.

Correct answer: B. Unbounded results risk blowing out context and cost; explicit pagination with a sensible default limit keeps individual calls manageable while still allowing access to more when needed.

Q60. A log-query tool that can return an unbounded number of log lines. How should this tool's design handle potentially very large result sets?

- A. ✓ Add pagination (page size and cursor/offset parameters) and a sensible default limit, rather than returning unbounded results.
- B. Let the tool always return every matching result in a single call.
- C. Cap results at a hardcoded small number with no way to retrieve more.
- D. Leave result size unbounded and rely on the model to only ask for what it needs.

Correct answer: A. Unbounded results risk blowing out context and cost; explicit pagination with a sensible default limit keeps individual calls manageable while still allowing access to more when needed.

Q61. A customer-list tool with no limit on how many customers it returns. How should this tool's design handle potentially very large result sets?

- A. Let the tool always return every matching result in a single call.
- B. Cap results at a hardcoded small number with no way to retrieve more.
- C. Leave result size unbounded and rely on the model to only ask for what it needs.
- D. ✓ Add pagination (page size and cursor/offset parameters) and a sensible default limit, rather than returning unbounded results.

Correct answer: D. Unbounded results risk blowing out context and cost; explicit pagination with a sensible default limit keeps individual calls manageable while still allowing access to more when needed.

Q62. A transaction-history tool that could return years of records at once. How should this tool's design handle potentially very large result sets?

- A. Cap results at a hardcoded small number with no way to retrieve more.
- B. Leave result size unbounded and rely on the model to only ask for what it needs.
- C. ✓ Add pagination (page size and cursor/offset parameters) and a sensible default limit, rather than returning unbounded results.
- D. Let the tool always return every matching result in a single call.

Correct answer: C. Unbounded results risk blowing out context and cost; explicit pagination with a sensible default limit keeps individual calls manageable while still allowing access to more when needed.

Q63. A document-search tool that can match a very large number of documents. How should this tool's design handle potentially very large result sets?

- A. Leave result size unbounded and rely on the model to only ask for what it needs.
- B. ✓ Add pagination (page size and cursor/offset parameters) and a sensible default limit, rather than returning unbounded results.
- C. Let the tool always return every matching result in a single call.
- D. Cap results at a hardcoded small number with no way to retrieve more.

Correct answer: B. Unbounded results risk blowing out context and cost; explicit pagination with a sensible default limit keeps individual calls manageable while still allowing access to more when needed.

Q64. An inventory tool that could return every sku in the catalog at once. How should this tool's design handle potentially very large result sets?

- A. ✓ Add pagination (page size and cursor/offset parameters) and a sensible default limit, rather than returning unbounded results.
- B. Let the tool always return every matching result in a single call.
- C. Cap results at a hardcoded small number with no way to retrieve more.
- D. Leave result size unbounded and rely on the model to only ask for what it needs.

Correct answer: A. Unbounded results risk blowing out context and cost; explicit pagination with a sensible default limit keeps individual calls manageable while still allowing access to more when needed.

Q65. A tool that can read and write customer records, currently sharing one broad api key across every use case. What's the better security practice?

- A. Keep one broad, shared credential across all tools for simplicity.
- B. Grant every tool full administrative access to avoid future permission issues.
- C. Rely on the system prompt to describe which permissions should be used for which tool.
- D. ✓ Scope each tool's credentials narrowly to only the permissions that specific tool actually needs.

Correct answer: D. Least-privilege credential scoping per tool limits the damage a misfiring or manipulated call can do — sharing broad credentials across tools with very different risk profiles is a common design mistake.

Q66. A tool that can issue refunds, currently using the same credentials as a read-only reporting tool. What's the better security practice?

- A. Grant every tool full administrative access to avoid future permission issues.
- B. Rely on the system prompt to describe which permissions should be used for which tool.
- C. ✓ Scope each tool's credentials narrowly to only the permissions that specific tool actually needs.
- D. Keep one broad, shared credential across all tools for simplicity.

Correct answer: C. Least-privilege credential scoping per tool limits the damage a misfiring or manipulated call can do — sharing broad credentials across tools with very different risk profiles is a common design mistake.

Q67. A tool that can delete resources, currently authenticated with the same key as a search tool. What's the better security practice?

- A. Rely on the system prompt to describe which permissions should be used for which tool.
- B. ✓ Scope each tool's credentials narrowly to only the permissions that specific tool actually needs.
- C. Keep one broad, shared credential across all tools for simplicity.
- D. Grant every tool full administrative access to avoid future permission issues.

Correct answer: B. Least-privilege credential scoping per tool limits the damage a misfiring or manipulated call can do — sharing broad credentials across tools with very different risk profiles is a common design mistake.

Q68. A tool that can modify user permissions, sharing credentials with an unrelated notification tool. What's the better security practice?

- A. ✓ Scope each tool's credentials narrowly to only the permissions that specific tool actually needs.
- B. Keep one broad, shared credential across all tools for simplicity.
- C. Grant every tool full administrative access to avoid future permission issues.
- D. Rely on the system prompt to describe which permissions should be used for which tool.

Correct answer: A. Least-privilege credential scoping per tool limits the damage a misfiring or manipulated call can do — sharing broad credentials across tools with very different risk profiles is a common design mistake.

Q69. A tool that can export bulk customer data, using the same broad credentials as a single-record lookup tool. What's the better security practice?

- A. Keep one broad, shared credential across all tools for simplicity.
- B. Grant every tool full administrative access to avoid future permission issues.
- C. Rely on the system prompt to describe which permissions should be used for which tool.
- D. ✓ Scope each tool's credentials narrowly to only the permissions that specific tool actually needs.

Correct answer: D. Least-privilege credential scoping per tool limits the damage a misfiring or manipulated call can do — sharing broad credentials across tools with very different risk profiles is a common design mistake.

Q70. A tool that can trigger production deploys, sharing a credential with a documentation-search tool. What's the better security practice?

- A. Grant every tool full administrative access to avoid future permission issues.
- B. Rely on the system prompt to describe which permissions should be used for which tool.
- C. ✓ Scope each tool's credentials narrowly to only the permissions that specific tool actually needs.
- D. Keep one broad, shared credential across all tools for simplicity.

Correct answer: C. Least-privilege credential scoping per tool limits the damage a misfiring or manipulated call can do — sharing broad credentials across tools with very different risk profiles is a common design mistake.

Q71. Consider a payment-charge tool that might be retried after a network timeout. Why does idempotent design matter here?

- A. Let duplicate effects happen and clean them up manually after the fact.
- B. ✓ Design the tool to be idempotent (e.g., via a client-supplied idempotency key) so a retried call doesn't duplicate the effect.
- C. Assume retries will never happen and design without idempotency in mind.
- D. Prevent all retries entirely, even for genuinely transient failures.

Correct answer: B. Retries are a normal part of resilient orchestration; without idempotency, a legitimate retry after a transient failure can silently duplicate a real-world effect like a charge or a shipment.

Q72. Consider an order-creation tool that could be called twice if a coordinator retries after an ambiguous failure. Why does idempotent design matter here?

- A. ✓ Design the tool to be idempotent (e.g., via a client-supplied idempotency key) so a retried call doesn't duplicate the effect.
- B. Assume retries will never happen and design without idempotency in mind.
- C. Prevent all retries entirely, even for genuinely transient failures.
- D. Let duplicate effects happen and clean them up manually after the fact.

Correct answer: A. Retries are a normal part of resilient orchestration; without idempotency, a legitimate retry after a transient failure can silently duplicate a real-world effect like a charge or a shipment.

Q73. Consider an email-send tool that could be triggered again by a retry after a partial failure. Why does idempotent design matter here?

- A. Assume retries will never happen and design without idempotency in mind.
- B. Prevent all retries entirely, even for genuinely transient failures.
- C. Let duplicate effects happen and clean them up manually after the fact.
- D. ✓ Design the tool to be idempotent (e.g., via a client-supplied idempotency key) so a retried call doesn't duplicate the effect.

Correct answer: D. Retries are a normal part of resilient orchestration; without idempotency, a legitimate retry after a transient failure can silently duplicate a real-world effect like a charge or a shipment.

Q74. Consider an inventory-decrement tool that might be called twice due to a retry. Why does idempotent design matter here?

- A. Prevent all retries entirely, even for genuinely transient failures.
- B. Let duplicate effects happen and clean them up manually after the fact.
- C. ✓ Design the tool to be idempotent (e.g., via a client-supplied idempotency key) so a retried call doesn't duplicate the effect.
- D. Assume retries will never happen and design without idempotency in mind.

Correct answer: C. Retries are a normal part of resilient orchestration; without idempotency, a legitimate retry after a transient failure can silently duplicate a real-world effect like a charge or a shipment.

Q75. Consider a resource-provisioning tool that could be invoked again after an unclear error. Why does idempotent design matter here?

- A. Let duplicate effects happen and clean them up manually after the fact.
- B. ✓ Design the tool to be idempotent (e.g., via a client-supplied idempotency key) so a retried call doesn't duplicate the effect.
- C. Assume retries will never happen and design without idempotency in mind.
- D. Prevent all retries entirely, even for genuinely transient failures.

Correct answer: B. Retries are a normal part of resilient orchestration; without idempotency, a legitimate retry after a transient failure can silently duplicate a real-world effect like a charge or a shipment.

Q76. Consider a notification tool that might fire twice if a retry happens after a timeout. Why does idempotent design matter here?

- A. ✓ Design the tool to be idempotent (e.g., via a client-supplied idempotency key) so a retried call doesn't duplicate the effect.
- B. Assume retries will never happen and design without idempotency in mind.
- C. Prevent all retries entirely, even for genuinely transient failures.
- D. Let duplicate effects happen and clean them up manually after the fact.

Correct answer: A. Retries are a normal part of resilient orchestration; without idempotency, a legitimate retry after a transient failure can silently duplicate a real-world effect like a charge or a shipment.

Q77. Before rolling out a new internal MCP server before rolling it out to the whole team, what should happen?

- A. Ship it directly to all users and monitor for complaints afterward.
- B. Test it once against a single trivial example and consider it validated.
- C. Skip testing since MCP servers are assumed to work correctly by default.
- D. ✓ Test it against representative real-world queries and edge cases before rolling it out broadly, verifying both correct routing and correct behavior on failure.

Correct answer: D. Tool and MCP server changes benefit from deliberate testing against realistic queries and edge cases — the exam consistently favors verifying behavior over shipping and hoping.

Q78. Before rolling out a redesigned tool description meant to fix a misrouting problem, what should happen?

- A. Test it once against a single trivial example and consider it validated.
- B. Skip testing since MCP servers are assumed to work correctly by default.
- C. ✓ Test it against representative real-world queries and edge cases before rolling it out broadly, verifying both correct routing and correct behavior on failure.
- D. Ship it directly to all users and monitor for complaints afterward.

Correct answer: C. Tool and MCP server changes benefit from deliberate testing against realistic queries and edge cases — the exam consistently favors verifying behavior over shipping and hoping.

Q79. Before rolling out a new error-response format for an existing tool, what should happen?

- A. Skip testing since MCP servers are assumed to work correctly by default.
- B. ✓ Test it against representative real-world queries and edge cases before rolling it out broadly, verifying both correct routing and correct behavior on failure.
- C. Ship it directly to all users and monitor for complaints afterward.
- D. Test it once against a single trivial example and consider it validated.

Correct answer: B. Tool and MCP server changes benefit from deliberate testing against realistic queries and edge cases — the exam consistently favors verifying behavior over shipping and hoping.

Q80. Before rolling out a newly added pagination parameter on a search tool, what should happen?

- A. ✓ Test it against representative real-world queries and edge cases before rolling it out broadly, verifying both correct routing and correct behavior on failure.
- B. Ship it directly to all users and monitor for complaints afterward.
- C. Test it once against a single trivial example and consider it validated.
- D. Skip testing since MCP servers are assumed to work correctly by default.

Correct answer: A. Tool and MCP server changes benefit from deliberate testing against realistic queries and edge cases — the exam consistently favors verifying behavior over shipping and hoping.

Q81. Before rolling out a new authentication scheme for an existing MCP server, what should happen?

- A. Ship it directly to all users and monitor for complaints afterward.
- B. Test it once against a single trivial example and consider it validated.
- C. Skip testing since MCP servers are assumed to work correctly by default.
- D. ✓ Test it against representative real-world queries and edge cases before rolling it out broadly, verifying both correct routing and correct behavior on failure.

Correct answer: D. Tool and MCP server changes benefit from deliberate testing against realistic queries and edge cases — the exam consistently favors verifying behavior over shipping and hoping.

Q82. Before rolling out a tool schema change that renames several fields, what should happen?

- A. Test it once against a single trivial example and consider it validated.
- B. Skip testing since MCP servers are assumed to work correctly by default.
- C. ✓ Test it against representative real-world queries and edge cases before rolling it out broadly, verifying both correct routing and correct behavior on failure.
- D. Ship it directly to all users and monitor for complaints afterward.

Correct answer: C. Tool and MCP server changes benefit from deliberate testing against realistic queries and edge cases — the exam consistently favors verifying behavior over shipping and hoping.

Q83. A widely-used lookup tool needs to be replaced with an improved version. What's the safer rollout approach?

- A. Keep both versions forever with no deprecation signal at all.
- B. ✓ Deprecate it gradually: introduce the replacement alongside it, mark the old one deprecated in its description, and remove it only after callers have migrated.
- C. Delete the old tool immediately once a replacement exists.
- D. Silently change the old tool's behavior in place without any notice.

Correct answer: B. Abrupt removal or silent behavior changes break existing callers; a staged deprecation with a clear signal and migration window is the safer path for any widely-depended-on tool.

Q84. A payment-processing tool needs to be replaced with an improved version. What's the safer rollout approach?

- A. ✓ Deprecate it gradually: introduce the replacement alongside it, mark the old one deprecated in its description, and remove it only after callers have migrated.
- B. Delete the old tool immediately once a replacement exists.
- C. Silently change the old tool's behavior in place without any notice.
- D. Keep both versions forever with no deprecation signal at all.

Correct answer: A. Abrupt removal or silent behavior changes break existing callers; a staged deprecation with a clear signal and migration window is the safer path for any widely-depended-on tool.

Q85. A search tool with many active callers needs to be replaced with an improved version. What's the safer rollout approach?

- A. Delete the old tool immediately once a replacement exists.
- B. Silently change the old tool's behavior in place without any notice.
- C. Keep both versions forever with no deprecation signal at all.
- D. ✓ Deprecate it gradually: introduce the replacement alongside it, mark the old one deprecated in its description, and remove it only after callers have migrated.

Correct answer: D. Abrupt removal or silent behavior changes break existing callers; a staged deprecation with a clear signal and migration window is the safer path for any widely-depended-on tool.

Q86. A reporting tool embedded in several automated workflows needs to be replaced with an improved version. What's the safer rollout approach?

- A. Silently change the old tool's behavior in place without any notice.
- B. Keep both versions forever with no deprecation signal at all.
- C. ✓ Deprecate it gradually: introduce the replacement alongside it, mark the old one deprecated in its description, and remove it only after callers have migrated.
- D. Delete the old tool immediately once a replacement exists.

Correct answer: C. Abrupt removal or silent behavior changes break existing callers; a staged deprecation with a clear signal and migration window is the safer path for any widely-depended-on tool.

Q87. A core inventory-query tool needs to be replaced with an improved version. What's the safer rollout approach?

- A. Keep both versions forever with no deprecation signal at all.
- B. ✓ Deprecate it gradually: introduce the replacement alongside it, mark the old one deprecated in its description, and remove it only after callers have migrated.
- C. Delete the old tool immediately once a replacement exists.
- D. Silently change the old tool's behavior in place without any notice.

Correct answer: B. Abrupt removal or silent behavior changes break existing callers; a staged deprecation with a clear signal and migration window is the safer path for any widely-depended-on tool.

Q88. A shared notification tool needs to be replaced with an improved version. What's the safer rollout approach?

- A. ✓ Deprecate it gradually: introduce the replacement alongside it, mark the old one deprecated in its description, and remove it only after callers have migrated.
- B. Delete the old tool immediately once a replacement exists.
- C. Silently change the old tool's behavior in place without any notice.
- D. Keep both versions forever with no deprecation signal at all.

Correct answer: A. Abrupt removal or silent behavior changes break existing callers; a staged deprecation with a clear signal and migration window is the safer path for any widely-depended-on tool.

Q89. A team is considering connecting a third-party MCP server with broad filesystem access and no published security review. What's the right posture?

- A. Connect it with full trust since it's convenient to use.
- B. Grant it broad access up front and narrow permissions later if problems appear.
- C. Assume third-party MCP servers are inherently safe because they're publicly available.
- D. ✓ Treat it with real caution: review its actual permission footprint, prefer least-privilege scoping, and avoid granting it broader access than its stated purpose requires.

Correct answer: D. Third-party MCP servers are effectively external code with tool-calling access — the same least-privilege, review-before-trust discipline applied to any third-party integration should apply here.

Q90. A team is considering connecting a community-maintained MCP server requesting access to production credentials. What's the right posture?

- A. Grant it broad access up front and narrow permissions later if problems appear.
- B. Assume third-party MCP servers are inherently safe because they're publicly available.
- C. ✓ Treat it with real caution: review its actual permission footprint, prefer least-privilege scoping, and avoid granting it broader access than its stated purpose requires.
- D. Connect it with full trust since it's convenient to use.

Correct answer: C. Third-party MCP servers are effectively external code with tool-calling access — the same least-privilege, review-before-trust discipline applied to any third-party integration should apply here.

Q91. A team is considering connecting an unfamiliar MCP server that requests far broader permissions than its stated purpose needs. What's the right posture?

- A. Assume third-party MCP servers are inherently safe because they're publicly available.
- B. ✓ Treat it with real caution: review its actual permission footprint, prefer least-privilege scoping, and avoid granting it broader access than its stated purpose requires.
- C. Connect it with full trust since it's convenient to use.
- D. Grant it broad access up front and narrow permissions later if problems appear.

Correct answer: B. Third-party MCP servers are effectively external code with tool-calling access — the same least-privilege, review-before-trust discipline applied to any third-party integration should apply here.

Q92. A team is considering connecting a third-party MCP server with no clear maintainer or update history. What's the right posture?

- A. ✓ Treat it with real caution: review its actual permission footprint, prefer least-privilege scoping, and avoid granting it broader access than its stated purpose requires.
- B. Connect it with full trust since it's convenient to use.
- C. Grant it broad access up front and narrow permissions later if problems appear.
- D. Assume third-party MCP servers are inherently safe because they're publicly available.

Correct answer: A. Third-party MCP servers are effectively external code with tool-calling access — the same least-privilege, review-before-trust discipline applied to any third-party integration should apply here.

Q93. A team is considering connecting an MCP server whose source code isn't available for review before granting it access. What's the right posture?

- A. Connect it with full trust since it's convenient to use.
- B. Grant it broad access up front and narrow permissions later if problems appear.
- C. Assume third-party MCP servers are inherently safe because they're publicly available.
- D. ✓ Treat it with real caution: review its actual permission footprint, prefer least-privilege scoping, and avoid granting it broader access than its stated purpose requires.

Correct answer: D. Third-party MCP servers are effectively external code with tool-calling access — the same least-privilege, review-before-trust discipline applied to any third-party integration should apply here.

Q94. A team is considering connecting a third-party server requesting access to customer PII with no stated data-handling policy. What's the right posture?

- A. Grant it broad access up front and narrow permissions later if problems appear.
- B. Assume third-party MCP servers are inherently safe because they're publicly available.
- C. ✓ Treat it with real caution: review its actual permission footprint, prefer least-privilege scoping, and avoid granting it broader access than its stated purpose requires.
- D. Connect it with full trust since it's convenient to use.

Correct answer: C. Third-party MCP servers are effectively external code with tool-calling access — the same least-privilege, review-before-trust discipline applied to any third-party integration should apply here.

Q95. A payment tool performs a sensitive, potentially high-impact action. What logging practice should it follow?

- A. Log only that the tool was called, with no arguments or result detail.
- B. ✓ Log every call's arguments, the calling agent/session, and the result — creating an auditable trail for that sensitive action.
- C. Log only failures, since successful calls don't need a record.
- D. Skip logging to keep the tool's implementation simpler.

Correct answer: B. Sensitive or high-impact tools need a full auditable trail — arguments, caller identity, and result — not just a bare success/failure flag, so any issue can be traced after the fact.

Q96. A data-export tool performs a sensitive, potentially high-impact action. What logging practice should it follow?

- A. ✓ Log every call's arguments, the calling agent/session, and the result — creating an auditable trail for that sensitive action.
- B. Log only failures, since successful calls don't need a record.
- C. Skip logging to keep the tool's implementation simpler.
- D. Log only that the tool was called, with no arguments or result detail.

Correct answer: A. Sensitive or high-impact tools need a full auditable trail — arguments, caller identity, and result — not just a bare success/failure flag, so any issue can be traced after the fact.

Q97. A permission-change tool performs a sensitive, potentially high-impact action. What logging practice should it follow?

- A. Log only failures, since successful calls don't need a record.
- B. Skip logging to keep the tool's implementation simpler.
- C. Log only that the tool was called, with no arguments or result detail.
- D. ✓ Log every call's arguments, the calling agent/session, and the result — creating an auditable trail for that sensitive action.

Correct answer: D. Sensitive or high-impact tools need a full auditable trail — arguments, caller identity, and result — not just a bare success/failure flag, so any issue can be traced after the fact.

Q98. A customer-communication tool performs a sensitive, potentially high-impact action. What logging practice should it follow?

- A. Skip logging to keep the tool's implementation simpler.
- B. Log only that the tool was called, with no arguments or result detail.
- C. ✓ Log every call's arguments, the calling agent/session, and the result — creating an auditable trail for that sensitive action.
- D. Log only failures, since successful calls don't need a record.

Correct answer: C. Sensitive or high-impact tools need a full auditable trail — arguments, caller identity, and result — not just a bare success/failure flag, so any issue can be traced after the fact.

Q99. A deployment tool performs a sensitive, potentially high-impact action. What logging practice should it follow?

- A. Log only that the tool was called, with no arguments or result detail.
- B. ✓ Log every call's arguments, the calling agent/session, and the result — creating an auditable trail for that sensitive action.
- C. Log only failures, since successful calls don't need a record.
- D. Skip logging to keep the tool's implementation simpler.

Correct answer: B. Sensitive or high-impact tools need a full auditable trail — arguments, caller identity, and result — not just a bare success/failure flag, so any issue can be traced after the fact.

Q100. A bulk-delete tool performs a sensitive, potentially high-impact action. What logging practice should it follow?

- A. ✓ Log every call's arguments, the calling agent/session, and the result — creating an auditable trail for that sensitive action.
- B. Log only failures, since successful calls don't need a record.
- C. Skip logging to keep the tool's implementation simpler.
- D. Log only that the tool was called, with no arguments or result detail.

Correct answer: A. Sensitive or high-impact tools need a full auditable trail — arguments, caller identity, and result — not just a bare success/failure flag, so any issue can be traced after the fact.

Q101. Which MCP primitive best fits exposing a static reference document the model can read as needed?

- A. An MCP tool
- B. An MCP prompt
- C. A CLAUDE.md entry
- D. ✓ An MCP resource

Correct answer: D. MCP distinguishes tools (invocable actions), resources (readable data/content), and prompts (reusable templates) — matching the primitive to the actual use case keeps an MCP server's interface clean.

Q102. Which MCP primitive best fits exposing an action the model can invoke to change or fetch live state?

- A. An MCP prompt
- B. A CLAUDE.md entry
- C. ✓ An MCP tool
- D. An MCP resource

Correct answer: C. MCP distinguishes tools (invocable actions), resources (readable data/content), and prompts (reusable templates) — matching the primitive to the actual use case keeps an MCP server's interface clean.

Q103. Which MCP primitive best fits exposing a reusable prompt template other clients can select and fill in?

- A. A CLAUDE.md entry
- B. ✓ An MCP prompt
- C. An MCP resource
- D. An MCP tool

Correct answer: B. MCP distinguishes tools (invocable actions), resources (readable data/content), and prompts (reusable templates) — matching the primitive to the actual use case keeps an MCP server's interface clean.

Q104. Which MCP primitive best fits exposing a read-only configuration file for the model to consult?

- A. ✓ An MCP resource
- B. An MCP tool
- C. An MCP prompt
- D. A CLAUDE.md entry

Correct answer: A. MCP distinguishes tools (invocable actions), resources (readable data/content), and prompts (reusable templates) — matching the primitive to the actual use case keeps an MCP server's interface clean.

Q105. Which MCP primitive best fits exposing an operation that sends an email on the model's behalf?

- A. An MCP resource
- B. An MCP prompt
- C. A CLAUDE.md entry
- D. ✓ An MCP tool

Correct answer: D. MCP distinguishes tools (invocable actions), resources (readable data/content), and prompts (reusable templates) — matching the primitive to the actual use case keeps an MCP server's interface clean.

Q106. Which MCP primitive best fits exposing a curated, reusable instruction template for a common task?

- A. An MCP tool
- B. A CLAUDE.md entry
- C. ✓ An MCP prompt
- D. An MCP resource

Correct answer: C. MCP distinguishes tools (invocable actions), resources (readable data/content), and prompts (reusable templates) — matching the primitive to the actual use case keeps an MCP server's interface clean.

Q107. A tool wraps a third-party search API with a strict per-minute call limit. How should the tool handle hitting the rate limit?

- A. Disable the tool entirely rather than handling the rate limit gracefully.
- B. ✓ Implement rate-limit-aware backoff in the tool itself and return a structured, retryable error when the limit is hit.
- C. Let calls fail silently with no indication that a rate limit was hit.
- D. Retry immediately and repeatedly with no backoff until it succeeds.

Correct answer: B. Handling rate limits inside the tool — with backoff and a structured, retryable error — lets the calling coordinator make an informed decision instead of failing silently or hammering the limit.

Q108. A tool wraps an internal service that degrades under bursty tool-call traffic. How should the tool handle hitting the rate limit?

- A. ✓ Implement rate-limit-aware backoff in the tool itself and return a structured, retryable error when the limit is hit.
- B. Let calls fail silently with no indication that a rate limit was hit.
- C. Retry immediately and repeatedly with no backoff until it succeeds.
- D. Disable the tool entirely rather than handling the rate limit gracefully.

Correct answer: A. Handling rate limits inside the tool — with backoff and a structured, retryable error — lets the calling coordinator make an informed decision instead of failing silently or hammering the limit.

Q109. A tool wraps a payment gateway with published rate limits. How should the tool handle hitting the rate limit?

- A. Let calls fail silently with no indication that a rate limit was hit.
- B. Retry immediately and repeatedly with no backoff until it succeeds.
- C. Disable the tool entirely rather than handling the rate limit gracefully.
- D. ✓ Implement rate-limit-aware backoff in the tool itself and return a structured, retryable error when the limit is hit.

Correct answer: D. Handling rate limits inside the tool — with backoff and a structured, retryable error — lets the calling coordinator make an informed decision instead of failing silently or hammering the limit.

Q110. A tool wraps a shared internal database that slows down under concurrent load from many agent sessions. How should the tool handle hitting the rate limit?

- A. Retry immediately and repeatedly with no backoff until it succeeds.
- B. Disable the tool entirely rather than handling the rate limit gracefully.
- C. ✓ Implement rate-limit-aware backoff in the tool itself and return a structured, retryable error when the limit is hit.
- D. Let calls fail silently with no indication that a rate limit was hit.

Correct answer: C. Handling rate limits inside the tool — with backoff and a structured, retryable error — lets the calling coordinator make an informed decision instead of failing silently or hammering the limit.

Q111. A tool wraps an external weather API with a modest free-tier call quota. How should the tool handle hitting the rate limit?

- A. Disable the tool entirely rather than handling the rate limit gracefully.
- B. ✓ Implement rate-limit-aware backoff in the tool itself and return a structured, retryable error when the limit is hit.
- C. Let calls fail silently with no indication that a rate limit was hit.
- D. Retry immediately and repeatedly with no backoff until it succeeds.

Correct answer: B. Handling rate limits inside the tool — with backoff and a structured, retryable error — lets the calling coordinator make an informed decision instead of failing silently or hammering the limit.

Q112. A tool wraps a shared logging service that throttles high-volume callers. How should the tool handle hitting the rate limit?

- A. ✓ Implement rate-limit-aware backoff in the tool itself and return a structured, retryable error when the limit is hit.
- B. Let calls fail silently with no indication that a rate limit was hit.
- C. Retry immediately and repeatedly with no backoff until it succeeds.
- D. Disable the tool entirely rather than handling the rate limit gracefully.

Correct answer: A. Handling rate limits inside the tool — with backoff and a structured, retryable error — lets the calling coordinator make an informed decision instead of failing silently or hammering the limit.

Q113. A workflow composes a lookup tool followed by a calculation tool followed by a notification tool. What matters most for this chain to be reliable in production?

- A. Assume the chain will always succeed end-to-end with no failure handling.
- B. Combine all steps into one large tool to avoid composition issues.
- C. Let each tool run independently with no attention to how its output feeds the next step.
- D. ✓ Ensure each tool's output is well-structured enough to feed cleanly into the next tool's expected input, and handle failure at any stage explicitly rather than assuming the whole chain succeeds.

Correct answer: D. Composed tool chains are only as reliable as their weakest handoff — structured, predictable outputs between stages and explicit failure handling at each stage are what make the chain production-safe.

Q114. A workflow composes a validation tool followed by a submission tool. What matters most for this chain to be reliable in production?

- A. Combine all steps into one large tool to avoid composition issues.
- B. Let each tool run independently with no attention to how its output feeds the next step.
- C. ✓ Ensure each tool's output is well-structured enough to feed cleanly into the next tool's expected input, and handle failure at any stage explicitly rather than assuming the whole chain succeeds.
- D. Assume the chain will always succeed end-to-end with no failure handling.

Correct answer: C. Composed tool chains are only as reliable as their weakest handoff — structured, predictable outputs between stages and explicit failure handling at each stage are what make the chain production-safe.

Q115. A workflow composes a search tool followed by a summarization tool followed by an export tool. What matters most for this chain to be reliable in production?

- A. Let each tool run independently with no attention to how its output feeds the next step.
- B. ✓ Ensure each tool's output is well-structured enough to feed cleanly into the next tool's expected input, and handle failure at any stage explicitly rather than assuming the whole chain succeeds.
- C. Assume the chain will always succeed end-to-end with no failure handling.
- D. Combine all steps into one large tool to avoid composition issues.

Correct answer: B. Composed tool chains are only as reliable as their weakest handoff — structured, predictable outputs between stages and explicit failure handling at each stage are what make the chain production-safe.

Q116. A workflow composes an authentication tool followed by a data-fetch tool that requires its token. What matters most for this chain to be reliable in production?

- A. ✓ Ensure each tool's output is well-structured enough to feed cleanly into the next tool's expected input, and handle failure at any stage explicitly rather than assuming the whole chain succeeds.
- B. Assume the chain will always succeed end-to-end with no failure handling.
- C. Combine all steps into one large tool to avoid composition issues.
- D. Let each tool run independently with no attention to how its output feeds the next step.

Correct answer: A. Composed tool chains are only as reliable as their weakest handoff — structured, predictable outputs between stages and explicit failure handling at each stage are what make the chain production-safe.

Q117. A workflow composes a data-cleaning tool followed by an analysis tool. What matters most for this chain to be reliable in production?

- A. Assume the chain will always succeed end-to-end with no failure handling.
- B. Combine all steps into one large tool to avoid composition issues.
- C. Let each tool run independently with no attention to how its output feeds the next step.
- D. ✓ Ensure each tool's output is well-structured enough to feed cleanly into the next tool's expected input, and handle failure at any stage explicitly rather than assuming the whole chain succeeds.

Correct answer: D. Composed tool chains are only as reliable as their weakest handoff — structured, predictable outputs between stages and explicit failure handling at each stage are what make the chain production-safe.

Q118. A workflow composes a draft-generation tool followed by a review tool followed by a publish tool. What matters most for this chain to be reliable in production?

- A. Combine all steps into one large tool to avoid composition issues.
- B. Let each tool run independently with no attention to how its output feeds the next step.
- C. ✓ Ensure each tool's output is well-structured enough to feed cleanly into the next tool's expected input, and handle failure at any stage explicitly rather than assuming the whole chain succeeds.
- D. Assume the chain will always succeed end-to-end with no failure handling.

Correct answer: C. Composed tool chains are only as reliable as their weakest handoff — structured, predictable outputs between stages and explicit failure handling at each stage are what make the chain production-safe.

End of Domain 4 deep-dive. This is an independent study aid; verify current exam logistics and blueprint details against your Claude Partner Network portal before sitting the exam.